Section 12.3: Designing for Global Catastrophic Failure

> ***Part of a 3-file series*** *covering global systems design (Sections 12.1–12.3).*

Executive Overview

Disaster recovery isn't optional for planet-scale systems. Global outages happen — AWS regions fail, undersea cables cut, cloud provider issues affect millions. The difference between resilient systems and academic exercises is a **tested, automated runbook** that executes under pressure.

This section covers: regional evacuation strategies with phase-by-phase runbooks, cross-region recovery plans for stateful services, DR drill frameworks, and production edge cases with interview-ready design patterns.

Regional Evacuation Strategies

What It Is

Regional evacuation is the process of safely migrating traffic and state from a failed region to healthy ones. The goal: preserve availability within SLA while respecting data residency constraints established in Section 12.2.

Core Mechanics

Full Evacuation Runbook:

```
Phase 1: Detection (1-5 minutes)
──────────────────────────────────────────────────
  - Health checks fail across 3+ AZs in the region
  - CloudWatch / Datadog alert fires to on-call engineer
  - Engineer confirms: not a false positive (check provider status page)
  - Decision gate: Initiate evacuation? YES / NO
  - If YES → trigger automated runbook

Phase 2: Preparation (5-15 minutes)
```

```
──────────────────────────────────────────────────────
  - Promote standby region database replica to primary
    (or verify active-active peer is healthy)
  - Verify replication lag at time of failure:
    - Lag < 10s → RPO acceptable, proceed
    - Lag > 30s → escalate to incident commander, assess data loss tolerance
  - Notify dependent services (internal APIs, partners)
  - Pre-warm caches in receiving region (replay recent cache-warming jobs)

Phase 3: Traffic Shift (0–30 minutes)
──────────────────────────────────────────────────────
  - Update DNS weights (requires TTL ≤ 30s, pre-reduced during prep):
      failed-region:   weight 0
      healthy-region:  weight 100
  - Monitor for 5-minute window:
      Error rate < 1%?  → Continue
      Error rate > 5%?  → Rollback immediately
  - Rollback criteria (automatic): >5% errors for 5 consecutive minutes

Phase 4: Validate (Ongoing)
──────────────────────────────────────────────────────
  - Confirm no data loss: checksum spot-check on promoted replica
  - Verify cross-region dependencies all resolved
  - Update incident status page
  - Begin failed region recovery in parallel (do not rush back)
```

DNS TTL Management — Pre-Evacuation:

```
Normal operation (performance optimized):
  TTL: 300s (5 minutes)
  Traffic shift time after DNS update: up to 5 minutes

Evacuation pre-step (must be done BEFORE incident):
  T-24h:  Reduce TTL to 30s
  T-0:    Update DNS record to healthy region
  T+30s:  ~95% of traffic on new record
          (residual 5% from long-cached resolvers may persist 1–2 min)

Why pre-reduce TTL?
  If you reduce TTL during the incident, the NEW short TTL takes 300s
  (the old TTL) to propagate to clients. You've gained nothing.
  TTL reduction must be planned ahead of time.
```

Detailed Comparison: Evacuation Strategies

Strategy	Recovery Time (RTO)	Data Loss (RPO)	Cost	Complexity
Backup & Restore	Hours	Last backup (hours)	$	Low

Strategy	Recovery Time (RTO)	Data Loss (RPO)	Cost	Complexity
Pilot Light	15–30 min	Seconds (if async repl)	l*Medium*ll* * *WarmStandby* * *l 2–10*min*l *Seconds*l$	Medium
Active-Active (Multi-Site)	Seconds	None (synchronous)	$$$$ l High l l **Active-Passive** l Minutes l Possible (async lag) l $$	Low

When to Use Each Strategy

Backup & Restore: Use only for non-critical, batch-oriented systems (data warehouses, dev/test environments). Unacceptable for any user-facing service with an SLA.

Pilot Light: Use for workloads with infrequent-but-real failure scenarios. A minimal "pilot" infrastructure (DNS config, DB replication) is always on; compute scales up only during evacuation. Good for: internal tools, low-traffic APIs.

Warm Standby: Use for production services where RTO of 10 minutes is acceptable. The standby runs at reduced capacity (e.g., 25% of primary), enough to handle degraded-mode traffic immediately.

Active-Active: Use for globally critical services where any downtime is unacceptable (payment processing, real-time communication). Every region handles live traffic; failover is instant (no "promotion" step needed). Requires the conflict resolution and consistency architecture from Section 12.2.

Cross-Region Recovery Plans

What It Is

Recovery plans must account for stateful services that cannot simply be restarted elsewhere. Database replication lag, in-flight message queue messages, and active user sessions all require coordinated recovery steps — in the right order.

Recovery Components

1. Database Recovery

```
Active-Passive scenario (us-east-1 fails, us-west-2 promoted):

Step 1: Identify replication lag at time of failure
  SELECT pg_last_xlog_receive_lsn() - pg_last_xlog_replay_lsn()
  AS replication_lag_bytes;

  → If lag > 0: some committed transactions on primary are lost
  → Document the exact LSN (log sequence number) for audit trail

Step 2: Promote standby
  pg_ctl promote -D /data/postgres
  (or: aws rds failover-db-cluster --db-cluster-identifier prod-cluster)

Step 3: Update application connection strings
  → DNS CNAME for db-primary.internal → us-west-2 endpoint
  → Applications reconnect automatically if using connection pooling

Step 4: Verify integrity
  → Run checksum on last 1000 transactions
  → Compare against any async audit log written to S3
```

2. Message Queue Recovery

```
Scenario: Kafka cluster in us-east-1 fails mid-message

Strategy — Dual-write during normal operation:
  Producer writes to: Kafka us-east-1 (primary)
                  AND Kafka us-west-2 (mirror)
  Consumer reads from: primary only (deduplicate by message ID)

On failover:
  Consumers reconnect to us-west-2
  Message ID deduplication prevents double-processing

Dead Letter Queue (DLQ) migration:
  DLQ messages from us-east-1 → copy to us-west-2 DLQ before traffic shifts
  Use: aws sqs send-message-batch (batch copy via Lambda)
  Without this step: Failed messages from the window before failure are lost
```

3. Session State Recovery

```
Option A — Stateless JWT tokens (recommended):
  Sessions are self-contained signed tokens
  No server-side state → survives any region migration automatically
  Trade-off: Cannot revoke tokens before expiry (use short TTL: 15 min + refresh)
```

```
Option B — Centralized session store:
  Redis Global Datastore: Active-active Redis across regions
  DynamoDB Global Tables: Eventual consistency (acceptable for sessions)
  On region failure: Existing sessions continue from other region's replica

Option C — Force re-authentication (fallback):
  If session store not available, expire all sessions
  Users re-authenticate against healthy region
  Acceptable for: Internal tools, non-real-time applications
  Unacceptable for: Mid-transaction flows (checkout, file upload)
```

Recovery Order Dependencies

Executing recovery steps out of order causes cascading failures. The correct sequence:

```
1. Database promoted first
   (Applications must have a writable DB before accepting traffic)

2. Message queue consumers reconnected
   (Prevents backlog buildup and duplicate processing)

3. Cache warming triggered
   (Prevents cold-cache thundering herd on receiving region)

4. Session store verified
   (Users should not be force-logged-out if avoidable)

5. DNS/load balancer updated LAST
   (Traffic only shifts after receiving region is fully ready)

✗
   Common mistake: Shifting DNS first, then fixing DB
   Result: Users hit healthy region, DB writes fail → 500 errors
   Resolution time: Much longer than the original outage
```

Disaster Recovery Drills

What It Is

Untested DR plans fail — not because the architecture is wrong, but because the runbook has undocumented dependencies, the automation hasn't been updated after the last infrastructure change, or the on-call engineer has never executed the steps under pressure.

The rule: If you haven't drilled it in the last quarter, your RTO is "unknown," not the number in your SLA document.

Drill Framework

Daily — Automated Chaos Experiments:

```
Scope: Single service, single AZ, off-peak hours (02:00–04:00 local)
Tools: AWS Fault Injection Simulator, Chaos Monkey, Gremlin
Examples:
  - Kill one of three app server instances → verify auto-scaling replaces it
  - Inject 200ms latency on DB connection → verify circuit breaker trips
  - Terminate a Redis node → verify cache fallback to DB
Rollback: Automated — chaos experiment stops if error rate > 1%
Output: Automated report; alert if experiment causes SLA breach
```

Monthly — Game Days:

```
Scope: Full subsystem (e.g., entire order processing pipeline)
Duration: 2–4 hours, business hours, with incident commander
Scenario examples:
  - "The us-east-1 RDS primary becomes unavailable"
  - "SQS in us-east-1 stops delivering messages"
  - "The CDN serving static assets is returning 503s globally"
Process:
  1. Announce scenario to on-call team
  2. Team executes runbook from memory (no coaching)
  3. Measure actual RTO achieved vs. target
  4. Document gaps; create follow-up tickets before end of day
```

Quarterly — Full Region Evacuation Drill:

```
Scope: Entire region (us-east-1) evacuated to us-west-2
Duration: Half day, non-production traffic first, then production
Steps:
  1. Pre-announce to customers (maintenance window)
  2. Execute full evacuation runbook (Phase 1–4 above)
  3. Run in us-west-2 for 2 hours minimum
  4. Fail back to us-east-1 (test this direction too!)
  5. Post-drill: Measure RTO achieved, data loss (RPO), any surprises

Target outcomes:
  - RTO achieved ≤ documented SLA
  - RPO = 0 for synchronous replicated data
  - Zero surprise dependencies discovered (all must be pre-documented)
  - Runbook updated with any corrections found during drill
```

Chaos Engineering Maturity Model

Level	Practice	What It Validates
1 — Unit	Kill single process	Auto-restart, health checks
2 — Service	Inject latency / errors	Circuit breakers, timeouts, retries
3 — Infrastructure	Terminate AZ	Multi-AZ architecture
4 — Region	Full region evacuation	DR runbook, RTO/RPO targets
5 — Provider	Simulate cloud provider outage	Multi-cloud / on-prem fallback

Most organizations operate at Level 2–3. Level 4 quarterly drills put you ahead of the majority of production systems.

Production Edge Cases

Cascading Failures

Scenario: us-east-1 fails. Traffic shifts to us-west-2. us-west-2 was at 60% capacity — now receives 160% of its designed load and begins failing.

```
Root cause: Active-passive with insufficient standby capacity
Detection signal: us-west-2 error rate climbs 30 seconds after traffic shift

Mitigations:
  1. Capacity planning: Standby must handle 100% of peak traffic (not just normal
traffic)
  2. Load shedding: us-west-2 sheds low-priority traffic (analytics, batch jobs)
     automatically when CPU > 80% — preserving capacity for core user flows
  3. Feature flags: Disable expensive features (personalization, recommendations)
     within 60s via feature flag service (LaunchDarkly, Unleash)
  4. Circuit breakers on downstream services: If payment processor also degrades
     under load, circuit opens and returns cached "try again later" response
     rather than queuing requests and amplifying the overload
```

Dependency Failures

Scenario: You evacuate your API servers from us-east-1 to us-west-2. But your third-party fraud detection service has no us-west-2 endpoint. Every transaction fails fraud check → 100% checkout failure rate in us-west-2.

```
Prevention:
  - Maintain a dependency map for every external service:
    Fraud API → supports: us-east-1, us-west-2, eu-west-1? YES/NO
  - For each external dependency: define degraded-mode behavior
    Fraud API unavailable → allow transaction (accept risk) OR reject all → which?
  - Test degraded mode quarterly: inject fraud API failure in game day

Recovery:
  - Activate fallback: in-house rule-based fraud scoring (pre-built, kept current)
  - Alert fraud team: manual review queue for transactions above $500
  - SLA impact: documented and pre-approved by risk team (not decided under
pressure)
```

Data Consistency During Extended Outage

Scenario: us-east-1 is down for 6 hours (not 6 minutes). Async replication to us-west-2 was lagging 45 seconds at time of failure. us-west-2 processed 6 hours of writes. Now us-east-1 comes back online.

```
The problem:
  us-west-2 has 6 hours of writes that us-east-1 missed
  us-east-1 has 45 seconds of writes that us-west-2 missed (pre-failure lag)

  These 45 seconds of writes must be reconciled against 6 hours of subsequent
writes

Resolution steps:
  1. Do NOT immediately re-enable us-east-1 as a peer
  2. Export the "orphaned" 45-second window WAL from us-east-1
  3. Apply conflict resolution rules to each orphaned write:
     - Financial transactions: Manual review; do not auto-resolve
     - User profile updates: LWW (us-west-2's newer version wins)
     - Inventory: Re-run business rules; flag oversold items
  4. Re-sync us-east-1 from us-west-2 (treat as a new replica)
  5. Promote us-east-1 back to active only after full sync confirmed

Key principle: Never re-introduce a previously-failed region as a peer
without a full reconciliation step. The temptation to "just turn it back on"
is the most common source of post-incident data corruption.
```

Capacity Mismatch on Failover

Scenario: Your receiving region does not have EC2 quota available to scale up for the incoming load. AWS returns “InsufficientInstanceCapacity” on auto-scaling.

```
Prevention:
  - Pre-warm standby capacity: Keep minimum instance count at 50% of primary
  - Use Capacity Reservations (AWS) or Committed Use Discounts (GCP) in standby
```

```
regions
  - Quarterly drill: Verify auto-scaling actually works (not just configured)

During incident:
  - Instance type flexibility: Configure ASG with 3–5 instance types
    (m5.xlarge, m5a.xlarge, m4.xlarge) → if one type unavailable, use another
  - Spot instances for non-critical workloads (batch jobs, analytics)
    to free on-demand capacity for core services
  - Hard shed non-essential services immediately to preserve capacity for core
```

Interview Design Problems

Problem 1: Design a DR Plan for a Global Payment Platform

Question: Your payment platform processes $10M/minute. The us-east-1 region goes down completely. Walk through your DR response from detection to full recovery.

Solution:

Pre-conditions (must be built before the incident): - Active-active architecture: us-east-1 and us-west-2 both handle live traffic at 60% capacity each. - Synchronous replication for transaction ledger (Spanner or CockroachDB), so RPO = 0. - DNS TTL pre-reduced to 30s on all production records. - Runbook automated: a single `./evacuate-region.sh us-east-1` command drives Phases 2–3.

Incident response timeline:

```
T+0:00  Health checks fail on us-east-1 (3 consecutive failures across 3 AZs)
T+0:01  PagerDuty fires; on-call engineer acknowledges
T+0:02  Engineer confirms: AWS Service Health confirms regional event
         Decision: Evacuate. Run: ./evacuate-region.sh us-east-1

T+0:03  Automated: us-east-1 weight → 0, us-west-2 weight → 100
         (us-west-2 already handling 60% load; now absorbs 100%)
         Load shedding activated: analytics, reporting APIs disabled

T+0:05  DNS propagation complete (TTL=30s)
         us-west-2 error rate: 0.3% (within SLA)

T+0:10  Cache warming complete (pre-warmed in parallel)
         us-west-2 p99 latency: 180ms (vs normal 120ms) — acceptable

T+0:15  All critical transaction flows confirmed healthy on us-west-2
         Status page updated: "us-east-1 degraded, service operational"

T+6:00  us-east-1 restored by AWS
T+6:30  Reconciliation: Confirm zero transaction data loss (synchronous repl)
```

```
T+7:00  Gradual traffic re-introduction: us-east-1 at 10%, then 50%, then 60%
T+8:00  Normal operation resumed
```

Key design decisions to call out in interview: 1. Active-active means no "promotion" step — RTO under 5 minutes. 2. Synchronous replication for ledger means RPO = 0 — no data loss, no reconciliation complexity. 3. Load shedding is pre-defined and automated — not a judgment call under pressure. 4. Re-introduction is gradual — never flip 100% traffic back instantly.

Problem 2: Designing Chaos Engineering for a New Service

Question: You are launching a new globally distributed microservice. How do you design a chaos engineering program to validate its DR properties before and after launch?

Solution:

Pre-launch validation (staging environment):

```
Week 1 — Unit chaos:
  Kill single service replica → verify k8s restarts within 30s
  Inject 500ms DB latency → verify circuit breaker trips, fallback activates
  Expected outcome: No user-visible errors during AZ-level events

Week 2 — Dependency chaos:
  Block all calls to downstream service X → verify degraded mode response
  Return 429 from rate-limited dependency → verify retry + backoff logic
  Kill Redis cache → verify graceful fallback to DB (slower but functional)

Week 3 — Region-level chaos:
  Simulate AZ failure (remove from LB target group) → verify traffic re-routes
  Simulate DB primary failure → verify replica promotion + reconnect < 30s
  Measure RTO achieved vs. target
```

Post-launch (production):

```
Month 1: Daily automated chaos (off-peak, AZ-level only)
Month 2: First monthly game day (subsystem-level, business hours)
Month 3: First quarterly drill (region evacuation)

Gate for each escalation:
  Only run higher-level chaos if lower levels pass consistently
  (Running region-level chaos when AZ-level chaos still causes issues = bad)
```

Metrics to track per experiment: - RTO achieved (time from failure injection to stable recovery) - Error rate during recovery window (should stay below SLA threshold) - Number of runbook gaps

discovered (target: 0 after 3 drills) - Mean Time to Detect (MTTD): time from failure injection to alert firing

---

Problem 3: Handling a "Silent" Data Divergence After Partial Outage

Question: After a 20-minute partial network partition between us-east-1 and eu-west-1, monitoring shows no alerts fired and traffic continued normally. Two days later, a customer reports their account balance is wrong. How do you investigate and remediate?

Solution:

Why this happens (root cause):

```
During partition (T=0 to T=20min):
  us-east-1 accepted writes for US users
  eu-west-1 accepted writes for EU users
  Neither region could confirm the other's writes

After partition healed (T=20min):
  Async replication resumed
  LWW conflict resolution silently discarded one side's writes
  No alert fired because: error rate = 0%, all requests succeeded
  The data loss was silent
```

Investigation:

```
Step 1: Identify the partition window
  Review VPC flow logs / network metrics for T=-5min to T=+25min around the report
  Confirm: cross-region replication lag spike during this window

Step 2: Extract diverged writes
  Query WAL / binary log for both regions during partition window
  Identify writes to the same keys from both regions

Step 3: Reconstruct correct state
  For financial balances: source-of-truth is the transaction log, not the balance
field
  Re-apply all transactions from partition window in causal order (use request
timestamps)
  Re-derive correct balance: balance = sum(all transactions for account)

Step 4: Reconcile
  Compare re-derived balance vs. current stored balance
  Difference = data lost to LWW conflict resolution
  Apply correcting journal entry (auditable, not a silent overwrite)
```

Prevention going forward:

```
1. Add conflict detection monitoring:
   After every replication cycle, check:
   any keys written in both regions during the lag window?
   Alert on ANY conflict detected (even if "resolved")

2. Switch from LWW to application conflict handlers for financial data
   (as covered in Section 12.2)

3. Implement reconciliation job:
   Daily: Re-derive all account balances from transaction log
   Alert if derived balance ≠ stored balance for any account
   This catches silent divergence within 24 hours, not 2 days
```

Key insight for interview: Silent failures are harder than loud ones. A system that fails loudly (errors, alerts) is easier to recover than one that silently accepts bad data. For financial systems, treat data correctness as a first-class monitoring concern — not just error rate and latency.